

Afterlight

A living record of the sight you fought to keep.

User Stories and Technical Specification

Product requirements document for the local-first personal eye record: who it is for, what they need, and how the system is built to meet those needs without breaking its safety and privacy invariants.

Document type	User stories + technical specification
Product	Afterlight (afterlight-eye)
Version of record	App v0.10.0 · Spec draft 1.0
Date	9 September 2026
Status	Draft for implementation reference — clinical wording still unreviewed
Audience	Product, engineering, clinical reviewers, future maintainers

This document is organisational. Afterlight does not diagnose, score risk, interpret imaging, or replace a clinician. Anything in this spec that appears to do so is a defect.

Contents

Use Word's Navigation Pane or Table of Contents field to browse. Part I covers user stories. Part II covers the technical specification.

- Part I — User Stories
 - 1. Product overview
 - 2. Problem and product thesis
 - 3. Personas
 - 4. Principles and non-negotiables
 - 5. Epic catalogue and user stories
 - 6. Out-of-scope requests (explicit refusals)
- Part II — Technical Specification
 - 7. Architecture overview
 - 8. Technology stack
 - 9. Data model
 - 10. Persistence, schema, and migrations
 - 11. Core domain modules
 - 12. Appointment brief engine
 - 13. Documents, imaging, and ingestion
 - 14. Handoff, export, and sharing
 - 15. Visualization engine and disease atlas
 - 16. Accessibility and low-vision requirements
 - 17. Performance budgets
 - 18. Privacy and security architecture
 - 19. Testing, quality gates, and release
 - 20. Known gaps and open items

Part I — User Stories

1. Product overview

Afterlight is a patient-owned, longitudinal eye-health record and visual diary. It captures what the patient experiences between clinic visits, keeps clinical evidence in the same chronology, and turns that continuous record into a one-page appointment brief.

It is local-first: no account, no server, no analytics, no telemetry. Records live in IndexedDB in the patient's browser. Data leaves the device only when the patient exports, prints, or shares it.

Primary surfaces:

- Today — the daily loop (under 30 seconds)
- What I See — patient-drawn visual field snapshots
- Timeline — derived chronology of every record type
- My Eyes — per-eye baselines, diagnoses, procedures, floaters, measurements
- Imaging & Documents — OCT, fundus, clinic letters, operative notes
- Appointments — prepare appointment brief, questions for the clinician
- Visualize — generic 3D eye, disease atlas, patient-view simulator
- Search & Ask — deterministic Q&A over the patient's own record
- Settings — themes, type scale, export/import, demo data, safety copy

2. Problem and product thesis

A patient experiences their sight continuously. Clinical care captures occasional snapshots. Between appointments the patient is the only person watching their eyes — and by the time they are in the chair, memory has collapsed into “I think it has been a bit worse?”

Afterlight's job is to close that gap: a continuous, eye-specific, dated account of what the patient saw, so that when a clinician asks “has it changed since last time?” the patient answers with a record instead of a guess.

Judging question for every feature: When the clinician asks “has it changed since last time?”, does the patient have a real answer?

3. Personas

3.1 Primary — Maya, post-retinal-detachment patient

Age 34. Retinal detachment and vitrectomy at 19. Persistent floaters in the right eye. Sees retina clinic twice a year. Anxious about new floaters and flashes. Wants a calm tool that helps her describe change without telling her she is going blind. Uses phone at night and laptop for appointment prep.

3.2 Secondary — Sam, bilateral AMD under monitoring

Age 68. Dry and wet AMD; injection series in the left eye. Low vision: needs large type and high contrast. Tracks Amsler checks at home and wants them comparable only when conditions match. Values print and fullscreen present mode over mobile share links.

3.3 Secondary — Dr Ravi, retina clinician (recipient, not user of account)

Has ten minutes. Will not open an app. Needs a one-page brief or a paper/PDF with clear provenance: what the patient reported versus what the clinic documented. Will not trust AI summaries of imaging.

4. Principles and non-negotiables

These are invariants. Breaking one is a defect regardless of feature quality. They are enforced in CI where possible (``npm run guard``, ``npm run nonetwork``).

1. Local-first, no backend. No account, no telemetry, no analytics, no third-party requests at runtime.
2. Provenance is never lost. Every record carries ``eye`` and ``source_type``. Patient-reported, patient-drawn, clinician-documented, device-measured and document-extracted content stay distinguishable in UI, briefs, print and export.
3. No diagnosis, no risk scores, no prognosis, no reassurance. The app organises and compares; it never interprets.
4. Missing data is shown as missing. “Not recorded” is never rendered as a normal result.
5. Nothing generic is ever presented as the patient’s own anatomy. Visualizations carry an unmissable educational boundary.
6. Calm. No alarm colour as decoration, no streak guilt, no frightening notifications.
7. The daily loop stays under 30 seconds.

5. Epic catalogue and user stories

Stories are grouped by epic. Acceptance criteria are written so they can be verified by running the app or a test, not by opinion.

Epic 1 — Daily record (Today)

US-1.1 — Log an unchanged day in one tap

As a person monitoring my eyes day to day, I want to record that nothing felt different today without filling a long form, so that six weeks of quiet days become evidence that a later change is new.

Priority: Must · **Acceptance criteria:**

- Today shows two large targets: “Nothing different today” and “Something changed”.
- Choosing “Nothing different today” creates a dated DailyLog (``overall: no_change``) and confirms on screen.
- No further fields are required.
- Today indicates that today’s check-in is already recorded after reload.

US-1.2 — Record a change against my usual baseline

As a person who notices a new or different symptom, I want to capture eye, symptom, comparison with usual, optional severity and free text, so that the appointment brief can say what is new without reconstructing memory.

Priority: Must · **Acceptance criteria:**

- Form appears only after “Something changed”.
- Eye is required (right / left / both).
- Symptom types include floaters, flashes, shadow/curtain, blur, glare, and others in the model catalogue.
- Baseline comparison is offered (same / slightly more / much more / fewer / different appearance / new).
- Severity 0–10 is optional and labelled as the patient’s own number.
- Suggestions come from the patient’s own history, never a generic list.

US-1.3 — Date an entry by when the change started

As a person writing up last night’s change this morning, I want to backdate the onset to yesterday or another day, so that the brief does not claim the change started a day later than it did.

Priority: Must · **Acceptance criteria:**

- Entry form offers Yesterday and an explicit date picker.
- Stored `date\_time` reflects onset, not typing time.

US-1.4 — See calm continuity without streak shame

As a person who sometimes forgets to log, I want to read one plain sentence about recent continuity, or nothing at all, so that gaps do not become guilt.

Priority: Must · **Acceptance criteria:**

- No streak flame, badge, or guilt copy.
- When there is a gap, messaging is equivalent to “Gaps are fine”.
- Continuity sentence is silent when there is nothing worth saying.

US-1.5 — Receive restrained urgent-assessment guidance

As a person who logs a sudden symptom commonly treated as urgent, I want to see one calm sentence pointing me at a real clinician, so that I am not left alone with a logging UI when I may need care.

Priority: Must · **Acceptance criteria:**

- Recording sudden floaters, flashes, curtain/shadow, or sudden vision drop triggers a short notice.
- Notice points to an ophthalmologist or emergency eye service.
- Notice does not estimate urgency, probability, or diagnosis.

Epic 2 — Visual diary (What I See)

US-2.1 — Draw what I see on a field canvas

As a person struggling to describe a floater or shadow in words, I want to place marks on a neutral visual-field canvas, so that a clinician can see the shape and location I cannot articulate.

Priority: Must · **Acceptance criteria:**

- Canvas supports pen, dot, strand, ring, blob, shadow, flash, blur tools with size and opacity.
- Drawing is tagged `source\_type: patient\_drawn` and a specific eye.

- Label states it is a patient-drawn representation, not a clinical retinal image.

US-2.2 — Compare two dates by overlay

As a person unsure whether a shadow has grown, I want to overlay two drawings from different dates, so that drift becomes visible instead of argued from memory.

Priority: Must · **Acceptance criteria:**

- Any two dated drawings for an eye can be overlaid or shown side by side.
- Difference is visual; no clinical verdict is stated.

US-2.3 — Get a text description of every drawing

As a screen-reader user or person preparing a brief, I want to read and search a plain-language description of my marks, so that drawings are accessible and appear in the brief as words.

Priority: Must · **Acceptance criteria:**

- Each drawing has an automatic text description (summary + mark-level detail).
- Description is used as alt text and in the appointment brief.

Epic 3 — Timeline

US-3.1 — See everything in one chronology

As a person with years of mixed records, I want to browse symptoms, drawings, scans, letters, appointments and procedures in date order, so that I can answer “what happened first?” without opening five folders.

Priority: Must · **Acceptance criteria:**

- Timeline is derived from entities; it is not a second stored copy.
- Rows show type, date, eye, and a provenance badge.
- Filters support eye, record type, and date range.
- Long histories window; earlier days load on demand.

Epic 4 — My Eyes

US-4.1 — Keep a separate baseline per eye

As a person whose eyes behave differently, I want to describe what “normal” is for each eye, with revision history, so that “worse than usual” has a defined referent.

Priority: Must · **Acceptance criteria:**

- Right and left profiles are independent.
- Baseline text and optional baseline drawing are supported.
- Revisions are dated.

US-4.2 — Track diagnoses, procedures, medications, measurements and prescriptions

As a person with a surgical and treatment history, I want to store structured clinical facts per eye with sources, so that the brief and My Eyes show the same coherent story.

Priority: Must · **Acceptance criteria:**

- Each entity carries eye + source_type + created/updated timestamps.
- Missing values show “Not recorded”, never “Normal”.
- Measurements are unit- and method-aware (e.g. IOP method, acuity notation).

US-4.3 — Track recurring floaters as objects

As a person with long-standing floaters plus occasional new ones, I want to name and update a persistent floater instead of logging fifty copies, so that “still present” is distinguishable from “genuinely new”.

Priority: Must · **Acceptance criteria:**

- FloaterObject has first_seen, appearance, status, baseline flag, drawing refs.
- New floater events can link to an object.

Epic 5 — Imaging and documents

US-5.1 — Store originals of scans and letters

As a person given PDFs and images by the clinic, I want to upload files and keep the original bytes, so that my archive matches what the clinic gave me.

Priority: Must · **Acceptance criteria:**

- Files stored as ArrayBuffer bytes in the `files` store.
- Imaging/document records link to stored files.
- Object URLs are released with `releaseFileURL`.

US-5.2 — Confirm metadata before it becomes fact

As a person importing a stack of files, I want to review extracted dates, eyes and types before they count, so that ambiguous dates and unread files never silently invent history.

Priority: Must · **Acceptance criteria:**

- Filename/PDF/DICOM hints are `document\_extracted` with `confirmed: false`.
- Ambiguous dates (e.g. 04-09-2026) offer both readings.
- Unconfirmed extractions stay out of the appointment brief.
- Unreadable files are stored with a plain explanation, not dropped.

US-5.3 — Compare two OCTs

As a person asked whether imaging changed, I want to view two OCT images side by side or with a slider, so that I can show the clinician both dates without re-exporting from the hospital portal.

Priority: Must · **Acceptance criteria:**

- Two scans of the same eye can be selected.
- Comparison is presentational; clinician interpretation is preferred text if present.
- No AI reading of the scan is presented as fact.

Epic 6 — Appointments and the brief

US-6.1 — Prepare a one-page appointment brief

As a person with a clinic visit next week, I want to generate a summary of what changed since the last visit, so that I can answer the clinician's first question in sixty seconds.

Priority: Must · **Acceptance criteria:**

- Brief covers a selectable period (since last visit, last 7/30 days, custom).
- Sections are ordered for a short read: per-eye changes (new / more than usual / unchanged / less), questions, drawings, clinical events.
- Patient-reported and clinician-documented content never share the same bullet list.
- Unconfirmed document extractions are excluded by default.

US-6.2 — Carry a running list of questions for my doctor

As a person who forgets questions between visits, I want to add, keep and attach answers to questions, so that the brief ends with what I actually want to ask.

Priority: Must · **Acceptance criteria:**

- Questions are first-class records with status (pending / asked / answered / follow-up).
- Questions appear on the brief.

US-6.3 — Put the brief on paper or a screen in the room

As a person handing information to a clinician, I want to save PDF, print, present fullscreen, or share a scoped extract, so that nothing depends on the clinician installing anything.

Priority: Must · **Acceptance criteria:**

- PDF is deterministic (same brief → same bytes) and uses base-14 fonts only.
- Print CSS prints the current page in black and white with provenance as words.
- Present mode pages sections at large type with wake lock.
- Share defaults to the patient's own entries and questions only; counts of inclusions and omissions are shown before a file exists.
- Every exported page carries a patient-generated footer.

Epic 7 — Search and Ask

US-7.1 — Search every record from the keyboard

As a person looking for “that glare thing in June”, I want to open ⌘K and search with optional field filters, so that I do not scroll five years of timeline.

Priority: Must · **Acceptance criteria:**

- Search covers symptoms, notes, documents, diagnoses, procedures, imaging metadata.
- Filters such as eye:left type:floaters after:2026-06 work.
- Clinical and everyday synonyms are symmetric (OCT ↔ tomography).
- Typo tolerance applies to long words only.

US-7.2 — Ask questions of my own record with citations

As a person who wants facts, not chat, I want to ask when something first appeared or how often it was recorded, so that I get an answer from my data or an honest not-found.

Priority: Must · **Acceptance criteria:**

- Answers are deterministic queries, not an LLM.
- Every answer cites source entries.
- Not-found is explicit: “I could not find that in your stored records.”
- No answer invents a number the record does not support.

Epic 8 — Education and visualization

US-8.1 — Explore generic anatomy and procedures

As a person trying to understand what a vitrectomy was, I want to open a clearly educational 3D model and procedure explainers, so that I can follow the clinic conversation.

Priority: Must · **Acceptance criteria:**

- Views carry GENERIC_MODEL_BOUNDARY wording.
- Personalisation (iris colour, pigmentation) is cosmetic only.
- Three.js loads only in the lazy Visualize chunk.

US-8.2 — Browse a condition atlas without being diagnosed

As a person researching a named condition, I want to navigate conditions and a patient-view simulator myself, so that I learn without the app ranking disease against my symptoms.

Priority: Must · **Acceptance criteria:**

- Atlas is never surfaced from the patient’s symptoms or ranked against their record.
- Field loss fades; it is never painted black.
- Retinal locations invert correctly to field locations (superior detachment → shadow from below).
- All atlas copy is flagged as unreviewed until clinical sign-off.

Epic 9 — Access, themes, and backup

US-9.1 — Use the app with low vision

As a person with reduced acuity or contrast sensitivity, I want to scale type, switch theme, and reduce glare/motion, so that I can complete the daily loop without a sighted helper.

Priority: Must · **Acceptance criteria:**

- Four themes including high-contrast light and dark.
- Type scale up to 200% with layout reflow and no horizontal overflow.
- Targets ≥44px; focus never removed; meaning not colour-only.
- Preferences persist in the record and survive export/import.

US-9.2 — Export and restore my whole record

As a person changing phone or afraid of clearing browser data, I want to export JSON (optionally encrypted) and import after preview, so that I am not trapped and I can keep two backups.

Priority: Must · Acceptance criteria:

- Export includes checksum, store counts and date span.
- Import shows a preview before writing; replace and merge modes exist.
- AES-GCM encryption with PBKDF2-SHA256 (250k iterations) is optional.
- Test-backup validates a file without writing.
- Calm weekly nudge if backup age is high; no nagging.

US-9.3 — Try demo data safely

As a person evaluating the app, I want to load a coherent synthetic history and remove it in one action, so that I can judge the product without risking my own record.

Priority: Must · Acceptance criteria:

- Demo records set `demo: true`.
- Demo data is removable in one action and never mixes into exports as personal history without the badge.

6. Out-of-scope requests (explicit refusals)

These are not backlog items. They are product refusals documented so they are not re-litigated as features.

- Diagnosing disease or ranking conditions against symptoms.
- Risk scores, probabilities, or “your vision is declining” messaging.
- Interpreting OCT or fundus images.
- Reassuring the patient that a quiet log means healthy eyes.
- Any backend account, cloud sync, or silent analytics.
- A general medical EMR or telemedicine platform.
- Bundling a runtime OCR/LLM that would require network calls or multi-megabyte downloads without an explicit future design review.

Part II — Technical Specification

7. Architecture overview

Afterlight is a single-page React application with no server. All persistence is browser IndexedDB. Derived views (timeline, indexes, briefs, search results) are computed from stored entities and never stored as a second source of truth.

High-level flow:

8. UI pages and components call the single store (`store.tsx`).
9. Store holds entity collections + meta; `data` identity is stable for caching.
10. Lib engines (brief, search, ask, trends, query, describe, share, pdf) are pure or near-pure over that data.
11. DB layer (`db.ts`) persists to IndexedDB stores; migrations run once inside a transaction after a backup snapshot.
12. Worker generates image thumbnails; service worker precaches the app shell for offline.
13. Nothing fetches from the network at runtime; `nonetwork` fails the build if that changes.

Logical modules:

- Presentation — pages/ (Today, WhatISee, Timeline, MyEyes, Imaging, Appointments, Visualize, Settings, SelfTests)
- State — lib/store.tsx, lib/router.ts (hash routes)
- Domain logic — lib/models.ts, brief.ts, search.ts, ask.ts, trends.ts, query.ts, describe.ts, education.ts, ingest.ts
- Persistence — lib/db.ts, migrations.ts, archive.ts
- Documents & handoff — pdf.ts, briefpdf.ts, briefexport.ts, share.ts, qr.ts
- Rendering — render.ts (visual field), engine/ (Three.js anatomy/atlas, lazy)
- Access & i18n — styles.css + tokens, lib/i18n.ts, locales/en.ts, print.css

8. Technology stack

Area	Choice
Runtime UI	React 18 + TypeScript
Build	Vite 7; code-split pages other than Today/Timeline
3D (lazy only)	three (Visualize chunk)
Storage	IndexedDB (afterlight db, version 3)
Drawing	Canvas 2D
Unit tests	Vitest + Testing Library + fake-indexeddb + axe-core
E2E	Playwright (Chromium desktop + WebKit iPhone 13)
Lint / a11y	ESLint 9 flat config + jsx-a11y as errors
CI gate	npm run verify = types + lint + guard + assets + changelog + tests + build + bundle + nonetwork

PWA	Service worker with real content-hashed precache; offline verified
PDF/QR	In-repo deterministic writers (lib/pdf.ts, lib/qr.ts) — no CDN fonts

Runtime dependencies beyond React and (for Visualize) three require explicit justification. Prefer a dozen lines of local code over a new package.

9. Data model

Defined in `app/src/lib/models.ts`. Every clinical/user record carries at least:

- `id`: string
- `created_at` / `updated_at`: ISO timestamps
- `eye`: right | left | both | not_applicable
- `source_type`: patient_reported | patient_drawn | clinician_reported | device_measurement | document_extracted | ai_generated
- `demo?`: true when synthetic

9.1 Core entity types

SymptomEntry: Daily symptom with `baseline_comparison`, optional severity, links to floater/drawing/appointment

DailyLog: Whole-day `no_change` or recorded check-in

FloaterObject: Persistent floater identity over time

VisualFieldDrawing: Marks on canvas + description + linked symptoms

Appointment: Clinic visit, actions, follow-up, linked docs/imaging/questions

DoctorQuestion: Running questions with status

Diagnosis: Documented diagnosis, not a symptom

Procedure: Surgery/laser/injection history

Medication / Prescription / Measurement: Treatments, glasses, unit-aware measurements

SelfTestResult: Home checks with mandatory test conditions

ImagingRecord / DocumentRecord: Scans and letters with file refs

StoredFile: `ArrayBuffer` bytes + size + `stored_at`

EyeBaseline: Per-eye normal description and revisions

GeneratedBrief: Saved brief artefacts

AppMeta: Preferences, schema version, backup age, display settings

9.2 Provenance rules

- UI badges distinguish patient vs clinician vs device vs extracted content everywhere it appears.
- Briefs and print never blend patient-reported and clinician-documented bullets in one list.

- `document_extracted` content requires `PERSON_CONFIRMED` before it is treated as fact or included in the brief.
- `ai_generated` is reserved for optional future summaries and is not used as a diagnosis channel.

10. Persistence, schema, and migrations

Database name ``afterlight``, `DB_VERSION 3`. Object stores (`keyPath id`): symptoms, dailyLogs, floaters, drawings, appointments, questions, diagnoses, procedures, medications, prescriptions, measurements, selfTests, imaging, documents, files, baselines, briefs, meta, backups.

- Schema changes bump `SCHEMA_VERSION` and ship a pure migration in `lib/migrations.ts`.
- Migrations run once inside a transaction after writing a pre-migration snapshot to ``backups``.
- Files are `ArrayBuffer` bytes, not Blobs, for reliable IndexedDB round-trip.
- Quota failures surface via `saveStoredFile` messaging — never silent loss.
- Archive export is v2: canonical JSON + SHA-256 checksum + per-store counts + date span.
- Import previews and validates; merge resolves conflicts by `updated_at` (newest wins).
- Multi-store writes use `dbWriteSnapshot` / serialised meta patches to avoid races.

11. Core domain modules

`store.tsx` — Single React store; stable data identity; indexes cached on data object.

`query.ts` — `select(data, type).eye(...).between(...).order(...)` over Phase-02 indexes.

`indexes.ts` — Timeline, per-day, per-eye/type maps, first-seen dates built once per data change.

`brief.ts` — What-changed engine: new / more than usual / unchanged / less, per eye.

`search.ts` — Field-scoped search, synonym table, typo tolerance, explainable ranking (``why``).

`ask.ts` — Deterministic intent grammar + citations; property test: no unsupported numbers.

`trends.ts` — Numeric series (acuity → logMAR); descriptive arithmetic only; no verdict language.

`describe.ts` — Drawing → words for screen readers, search, and briefs.

`suggestions.ts` — Quick-entry suggestions from patient history only.

`streak.ts` — Optional calm continuity sentence; no gamification.

`ingest.ts` — Filename/PDF/DICOM metadata as unconfirmed suggestions.

`archive.ts` — Export/import envelope usable even if UI crashed (error boundary escape hatch).

12. Appointment brief engine

Inputs: store data + period start/end + optional sections + patient header fields. Output: a document model consumed by UI, print CSS, and `lib/pdf.ts`.

14. Select entities in period via query layer (not raw array scans).
15. Split by eye; classify symptom change vs baseline comparison and status.
16. Separate patient-reported vs clinician/device/extracted blocks.
17. Exclude `confirmed=false` document extracts.
18. Include drawings (rasterised for PDF), clinical events, current treatment, questions.
19. Optional opt-in sections: recorded numbers, home checks — off by default to keep one page.

20. Render: on-screen layout, print.css, deterministic PDF, present mode, share extract.

Determinism: pdf.ts forbids Date.now, Math.random and locale formatting (guard-enforced). Fonts are PDF base-14. Same brief bytes allow “has this changed?” by file compare.

13. Documents, imaging, and ingestion

- Upload path preserves original bytes; thumbnails generated in a worker (WebP/JPEG).
- Ingestion heuristics: filename tokens, `\_OD`/`\_OS`, PDF header/page count, DICOM study date/modality/laterality/institution/encapsulated JPEG.
- Ambiguous dates return both readings; never guess locale.
- OCR is a registered-recogniser seam only; engine is not bundled (docs/ocr.md) because runtime fetch would break non-negotiable #1 and bundling costs ~10MB.
- OCT comparison UI is presentational only.

14. Handoff, export, and sharing

- Three exit paths only: print, generated PDF, encrypted extract — all produced locally.
- Share: range-scoped AES-GCM bundle + descriptor; default scope = patient entries + questions; imaging/documents/diagnoses/original files require explicit inclusion; counts of included and omitted items shown before file creation.
- Passphrase: four-word phrase for verbal transfer; never sent with the file.
- QR card (lib/qr.ts): byte-mode versions 1–20, levels L/M; plain text and labelled as such.
- PATIENT_GENERATED_FOOTER on every page of print/PDF.
- clinician-note.md links from the brief for the clinician reader (needs human sign-off).

15. Visualization engine and disease atlas

- engine/ is framework-agnostic Three.js under app/src/engine/.
- Anatomy dimensions from engine/anatomy/dimensions.ts in millimetres.
- Procedural iris/fundus/vessels; parameter deltas for conditions (atlas), not separate baked pictures per severity.
- Patient-view simulator: scotoma, metamorphopsia, field loss, curtain, floaters, glare, etc.
- Enforced properties: field loss fades and is never black; retinal location inverts to field.
- Standalone single-file build EyeExplorer-engine.html ≤5MB, offline-capable.
- GENERIC_MODEL_BOUNDARY on every view including canvas accessible name.
- Blender bake track documented but not run in this environment (assets stay procedural).

16. Accessibility and low-vision requirements

- Sizes from --fs-* tokens only; colours from theme tokens; contrast.test.ts must pass in all four themes.
- Interactive targets ≥44px; keyboard journey tested; focus trap/restore in modals.
- axe runs over every page, dialog, theme, and largest type scale.
- prefers-reduced-motion plus explicit reduced-motion / glare-comfort / dimmed-imagery preferences.
- Canvas and charts have real text equivalents (describe.ts + table view of chart data).

- Voice entry deliberately not shipped (browser Speech API would send audio to third parties); OS dictation supported.
- Known WebKit offline-reload E2E limitation documented for manual device check.

17. Performance budgets

- Initial JS gzip ≤ 120 KB (enforced by npm run bundle).
- Initial CSS gzip ≤ 20 KB.
- Pages other than Today and Timeline lazy-loaded; guard fails if a heavy page creeps into the entry chunk.
- Timeline windows (~60 days) with explicit “show earlier”.
- Indexed caches: ~10-year synthetic record (20k symptoms, 500 drawings, 200 scans) — index build ~12ms, search ~22ms cold / ~4ms warm, brief ~5ms (benchmarks in tests, budgets looser to catch quadratic regressions).
- Object URL leak guards with releaseFileURL thresholds.

18. Privacy and security architecture

- No remote endpoints. nonetwork audits built dist/ and site/ for src/href/url()/@import/workers/sockets.
- Anchor links a user chooses are allowed; resource loads are not.
- Threat model prioritises data integrity (record is unrecoverable from elsewhere) — see SECURITY.md.
- Encryption optional on export; no recovery if passphrase forgotten (honest trade).
- No medical advice in issues (CODE_OF_CONDUCT); clinical accuracy issues go to a human template.

19. Testing, quality gates, and release

- Unit: domain invariants, migrations, brief, ask property tests, acuity conversions, contrast, a11y, keyboard, smoke pages.
- E2E: daily loop, backdating, persistence, mobile tab bar/sheets, offline, zero third-party requests, engine.
- Gate: npm run verify must be clean before any phase is reported done.
- Release: npm run release refuses to tag while docs/clinical-review.md status is NOT REVIEWED.
- Version $< 1.0.0$ until clinical wording is reviewed; CHANGELOG generated from lib/changelog.ts.
- Publishing (Pages workflow) is workflow_dispatch only — human decision.

20. Known gaps and open items

Clinical review: All condition profiles, atlas “what people notice” lines, home-check instructions, clinician-note.md, printed brief wording need ophthalmologist/optometrist sign-off. Pack: npm run clinical-pack.

OCR engine: Seam exists; engine not bundled (cost + offline purity). See docs/ocr.md.

Blender bake track: Asset pipeline written; Blender not installed in the build environment; engine remains procedural.

Physical print check: A4 and Letter paper verification outstanding.

Clinician-reader test: Someone unfamiliar with the app should read the brief against a clock.

Real-device PWA install: iPhone and Android install + iOS offline reload need manual verification.

i18n coverage: Shell, daily loop, provenance and safety strings extracted; deeper pages not fully extracted — documented honestly in docs/translating.md.

Hosted deploy: Intentionally not automatic; human publish only.

21. Definition of done (for features and phases)

- npm run verify passes clean.
- New behaviour has a test that would fail if it regressed.
- Keyboard and screen-reader paths work for anything new.
- No non-negotiable regressed — checked explicitly.
- Demo dataset still loads and looks coherent.
- Docs updated where a user or future agent would need to know.
- Clinical copy changes flagged for human review; agents never self-approve clinical wording.

End of specification. Source of truth for implementation remains the repository (AGENTS.md, docs/plan/, lib/models.ts). This document summarises product intent and architecture for stakeholders; it does not replace the phase briefs or the clinical review.